

Guide Administrateur SILLON

SILLON — Guide d'installation administrateur

Ce guide décrit l'installation de SILLON sur un serveur cible, à destination des administrateurs de la direction. Il complète le [SILLON cahier des charges.md](#) (choix fonctionnels et techniques) et le [README.md](#) (vue d'ensemble, état du projet).

1. Prérequis

1.1 Système cible

- **Debian 13 (Trixie)**, architecture amd64.
- Accès réseau **limité aux dépôts Debian via le proxy d'entreprise** : aucun accès à internet en général, ni à un registre de conteneurs (Docker Hub ou équivalent). Cette contrainte est structurante — voir §4.
- Compte d'installation avec droits root (ou sudo).

1.2 Réseau

- Le serveur doit être joignable en HTTPS (443) depuis les postes des agents. Le port 80 est utilisé uniquement pour la redirection vers 443.
- Aucun port applicatif (API PostgREST sur 3000, orchestrateur sur 5000, PostgreSQL sur 5432) n'est exposé au-delà de localhost : tout le trafic externe transite par Nginx.

1.3 Paquets Debian requis

Les dépendances sont déclarées dans chaque paquet SILLON et installées automatiquement par apt/dpkg si le dépôt Debian est accessible : postgresql-17, nginx, openssl, sudo, fail2ban, python3-flask, python3-psycopg2, python3-jwt, gunicorn, podman. Aucune action manuelle requise en amont.

2. Vue d'ensemble des paquets

Paquet	Rôle	Dépend de
sillon-server	Socle : PostgreSQL, Nginx, API de requêtage (PostgREST), catalogue applicatif, anti-bruteforce	—
sillon-orchestrateur	Création/suppression des bases, SQL libre, import CSV, partage, file d'attente	sillon-server
sillon-worker	Consommation de la file, lancement des conteneurs d'exécution de scripts	sillon-orchestrateur

Paquet	Rôle	Dépend de
sillon-image-execution	Image figée (Python/R) utilisée pour l'exécution des scripts	sillon-worker
sillon-tutoriel (<i>optionnel</i>)	Jeu de données réel et tutoriel PDF, partagés automatiquement avec tous les comptes — VM de démo/formation uniquement, voir §7.2	sillon-image-execution
sillon-demo-sirene (<i>optionnel</i>)	Jeu de données Sirene massif (~43,9 millions de lignes) et scripts de démonstration à l'échelle — VM de démo/formation uniquement, voir §7.2	sillon-tutoriel
sillon-purge (<i>optionnel, recommandé en production</i>)	Purge automatique quotidienne des résultats de jobs, dépôts temporaires orphelins et journal d'audit — voir §7.4	sillon-server

L'ordre d'installation ci-dessus est obligatoire : chaque paquet vérifie au `postinst` que le précédent est déjà configuré (présence de `/etc/sillon/secrets.env`, service actif) et refuse de s'installer sinon.

3. Obtention des paquets

Deux cas de figure :

- **Paquets déjà construits** : le dossier `build/` du dépôt contient les `.deb` prêts à l'emploi (`sillon-server_0.1.0_amd64.deb`, `sillon-orchestrateur_0.1.0_all.deb`, `sillon-worker_0.1.0_all.deb`, `sillon-image-execution_0.1.0_amd64.deb`, et les deux paquets optionnels `sillon-tutoriel_0.1.0_all.deb` et `sillon-demo-sirene_0.1.0_all.deb` — §7.2). C'est le cas le plus courant : passer directement à l'étape 4.
- **Reconstruction nécessaire** (nouvelle version, modification du code applicatif) : depuis une machine **ayant accès à internet** (le binaire PostgREST et l'image d'exécution sont téléchargés/construits à cette étape, pas sur la cible) :

```
cd build/
./build.sh
```

Le script vendorise le binaire PostgREST officiel (vérifié par somme SHA256) et construit l'image Podman d'exécution, avant de produire les quatre `.deb`. Voir cahier des charges §12.2 à §12.5 pour le détail.

Transférer ensuite les quatre `.deb` vers le serveur cible (SCP, dépôt APT interne, ou tout autre moyen).

4. Pourquoi cette contrainte de vendoring ?

Point important à comprendre avant d'installer : le serveur cible **n'a jamais accès à un registre de conteneurs ni à internet**, seulement aux dépôts Debian via le proxy d'entreprise. Un `podman build` ou un téléchargement direct lancés au moment de l'installation échoueraient systématiquement (constaté lors des premiers tests de déploiement réels).

C'est pourquoi :

- le binaire PostgREST est embarqué tel quel dans `sillon-server` (`/usr/lib/sillon/postgrest`) ;
- l'image d'exécution est construite en amont (hors cible) et embarquée en archive dans `sillon-image-execution` (`/usr/lib/sillon/image-execution.tar`), chargée localement via `podman load` au `postinst` — jamais reconstruite sur place.

Aucune action requise de l'administrateur sur ce point : c'est purement informatif, pour comprendre pourquoi une tentative de reconstruction sur la cible échouerait.

5. Installation

Sur le serveur cible, dans l'ordre :

```
sudo dpkg -i sillon-server_0.1.0_amd64.deb
sudo apt-get install -f      # résout les dépendances manquantes si besoin
```

```
sudo dpkg -i sillon-orchestrateur_0.1.0_all.deb
sudo dpkg -i sillon-worker_0.1.0_all.deb
sudo dpkg -i sillon-image-execution_0.1.0_amd64.deb
```

Sur une VM de démonstration ou de formation uniquement, deux paquets optionnels peuvent ensuite être installés (§7.2) :

```
sudo dpkg -i sillon-tutoriel_0.1.0_all.deb      # jamais sur un déploiement de
production
sudo dpkg -i sillon-demo-sirene_0.1.0_all.deb   # optionnel, nécessite un accès
Internet - voir §7.2
```

Ce que fait chaque `postinst`, en résumé

- `sillon-server` : génère les secrets applicatifs (`/etc/sillon/secrets.env`, `chmod 600 root:root`), crée la base `sillon_catalog` et y déploie le catalogue (`schema.sql`), **dimensionne le moteur PostgreSQL selon la RAM et le nombre de cœurs détectés** (`shared_buffers`, `effective_cache_size`, `maintenance_work_mem`, parallélisation, points de contrôle, `pg_stat_statements` — §7.1, redétecté et réappliqué sans effet si déjà correct à chaque réinstallation), configure l'API PostgREST (`/etc/sillon-api.conf`), génère un certificat TLS auto-signé si aucun n'est présent (`/etc/ssl/sillon/`), active le site Nginx, configure la limitation de débit et Fail2Ban sur `/api/rpc/login`, met en place la purge automatique du journal d'audit, et démarre les services `sillon-api` et `nginx`.

À la première installation, l'identifiant et le mot de passe administrateur initiaux sont écrits dans `/root/sillon-admin-initial.txt` (`chmod 600 root:root`), jamais affichés en clair sur la sortie du `postinst` — celle-ci se contente d'indiquer l'emplacement du fichier :

```
=== Identifiants administrateur initiaux écrits dans /root/sillon-admin-
initial.txt (lecture root uniquement) ===
=== À consulter puis à changer dès la première connexion ===
```

Consulter ce fichier immédiatement (`sudo cat /root/sillon-admin-initial.txt`), se connecter, changer le mot de passe depuis le panneau d'administration, puis **supprimer le fichier** (`sudo rm /root/sillon-admin-initial.txt`). Ce mot de passe n'est stocké nulle part ailleurs en clair.

(Avant la version 0.1.25 de `sillon-server`, ce mot de passe s'affichait directement dans la sortie du `postinst` — corrigé car cette sortie est généralement journalisée durablement par `apt/systemd`, contrairement au fichier `root-only` ci-dessus.)

- `sillon-orchestrateur` : applique les droits sur `/opt/sillon-orchestrateur`, active et démarre le service `sillon-orchestrateur` (Gunicorn, port 5000 en local).
- `sillon-worker` : configure Podman en mode rootless pour l'utilisateur système `www-data` (répertoire `$HOME` dédié, gestionnaire de cgroups `cgroupfs`, plage `subuid/subgid`, lingering `systemd`), autorise la connexion PostgreSQL depuis les conteneurs de script (`pg_hba.conf`), puis active et démarre le service `sillon-worker`.
- `sillon-image-execution` : charge l'image d'exécution vendorisée dans le stockage Podman rootless de `www-data`.

Une mise à jour ultérieure (paquets déjà installés)

Les `postinst` détectent une installation existante (présence de la base `sillon_catalog`) et n'effectuent alors que des migrations incrémentales — **aucune recréation destructive**, secrets et certificat TLS institutionnel conservés tels quels.

6. Vérifications post-installation

```
# Services actifs
systemctl status sillon-api sillon-orchestrateur sillon-worker nginx postgresql
fail2ban
```

```
# Journaux en cas de problème
journalctl -u sillon-api -n 50
journalctl -u sillon-orchestrateur -n 50
journalctl -u sillon-worker -n 50
```

Puis, depuis un navigateur : se connecter à `https://<adresse-du-serveur>/` avec l'identifiant administrateur initial noté à l'étape 5, et changer immédiatement ce mot de passe depuis le panneau d'administration.

Le certificat TLS généré par défaut est **auto-signé** (donc affiché comme non fiable par le navigateur). Le remplacer par un certificat institutionnel avant toute mise en production réelle :

```
sudo cp mon-certificat.crt /etc/ssl/sillon/sillon.crt
sudo cp ma-cle.key /etc/ssl/sillon/sillon.key
sudo chown root:root /etc/ssl/sillon/sillon.crt
sudo chown root:www-data /etc/ssl/sillon/sillon.key
sudo chmod 644 /etc/ssl/sillon/sillon.crt
sudo chmod 640 /etc/ssl/sillon/sillon.key
sudo systemctl reload nginx
```

(Un `postinst` ultérieur ne régénère jamais ce certificat s'il est déjà présent — le remplacement ci-dessus est donc définitif jusqu'à intervention manuelle.)

7. Configuration optionnelle

7.1 Notifications par mail

Tout traitement mis en file d'attente (import CSV volumineux, requête SQL basculée en tâche de fond, script Python/R, suppression de base) peut notifier son auteur par mail à la fin — l'utilisateur en décide lui-même à chaque

lancement, via une fenêtre de confirmation qui propose aussi de joindre le résultat au mail (uniquement si le fichier fait moins de 5 Mo ; au-delà, ou si la case n'est pas cochée, le mail renvoie simplement vers l'onglet Suivi de l'application). Il n'y a donc rien à activer côté administrateur pour que la fonctionnalité existe : seul le réglage du relais SMTP sortant reste à sa charge.

Par défaut, l'orchestrateur et le travailleur de file d'attente (`sillon-worker`) tentent chacun d'envoyer leurs notifications via un relais SMTP local (`localhost:25`), sans authentification — ce qui ne fonctionne que si un agent de transport de courrier tourne réellement sur la machine. Pour utiliser le relais SMTP interne de la direction (un relais ouvert, sans authentification ni TLS ; SILLON ne gère pas de couple utilisateur/mot de passe SMTP), deux façons de procéder, équivalentes et modifiables à chaud sans redémarrage de service :

- **Panneau d'administration** (recommandé) : onglet Administration, section « Notifications par mail » — renseigner le serveur SMTP (hôte), le port et l'adresse d'expéditeur, puis « Enregistrer ». Le bouton « Envoyer un mail de test » déclenche un envoi immédiat à l'adresse du compte administrateur connecté, avec les valeurs du formulaire (même non encore enregistrées) — pratique pour valider un réglage avant de le sauvegarder.
- **SQL direct**, depuis le catalogue applicatif (`sillon_catalog`) :

```
UPDATE public.parametres SET valeur = 'smtp.exemple-direction.fr' WHERE cle
= 'smtp_hote';
UPDATE public.parametres SET valeur = '25'                                WHERE cle
= 'smtp_port';
UPDATE public.parametres SET valeur = 'sillon@exemple-direction.fr' WHERE
cle = 'smtp_expeditteur';
```

Le lien inclus dans chaque mail (vers l'onglet Suivi de l'application) est construit à partir d'une adresse de base distincte, elle **non modifiable à chaud** : la variable d'environnement `SILLON_URL` (`https://localhost` par défaut). **Elle doit être réglée séparément sur les deux services**, `sillon-orchestrateur` (import CSV, requête SQL, suppression de base) et `sillon-worker` (scripts Python/R) — ce sont deux unités systemd indépendantes, chacune avec son propre `Environment=` ; régler l'une sans l'autre laisse les mails de fin de script pointer vers l'ancienne adresse alors que les autres notifications sont correctes (piège constaté en le réglant une seule fois par erreur) :

```
# systemctl edit sillon-orchestrateur, section [Service]
Environment=SILLON_URL=https://sillon.exemple-direction.fr
# systemctl edit sillon-worker, section [Service]
Environment=SILLON_URL=https://sillon.exemple-direction.fr
```

Puis `systemctl restart sillon-orchestrateur sillon-worker`. Un échec d'envoi de notification est journalisé (`journalctl -u sillon-orchestrateur` ou `-u sillon-worker` selon le type de traitement) mais ne bloque jamais le traitement lui-même — le résultat reste consultable et téléchargeable depuis l'onglet Suivi, notification par mail ou non.

7.2 Compte de démonstration et tutoriel

Deux façons de peupler un compte de démonstration, **jamais installées automatiquement avec le socle applicatif** — jamais sur un serveur de production :

- `sillon-tutoriel` (recommandé pour une VM de démonstration/formation) : cinquième paquet `.deb`, optionnel, à installer après les quatre autres (`sudo dpkg -i sillon-tutoriel_0.1.0_all.deb`). Son `postinst` crée (ou réutilise) un compte technique `demo@sillon.local` (profil agent, mot de passe généré aléatoirement — voir plus bas) et importe un **jeu de données réel** : les 34 868 communes de France et leurs 18 régions (source data.gouv.fr, INSEE/IGN, Licence Ouverte 2.0), avec des requêtes d'exemple déjà dans l'historique et deux scripts

Python/R déjà exécutés. Fournit en plus un **tutoriel PDF complet** (exercices SQL en difficulté croissante, puis formation avancée Python et R), installé dans `/usr/share/doc/sillon-tutoriel/` et publié aux côtés de la documentation existante sur <https://<serveur>/Documentation/>.

Ce jeu de données est partagé automatiquement avec tous les comptes, existants comme à venir (lecture seule, exécution de scripts Python/R autorisée) : chaque agent le retrouve dans son propre onglet Bases, sous le nom du compte `demo@sillon.local`, sans jamais avoir besoin de s’y connecter. Le tutoriel PDF et ses corrigés (`corriges-tutoriel.zip`) sont de la même façon visibles depuis la modale « À propos » de **tout** compte, dès lors que ce paquet est installé — chacun suit le tutoriel avec sa propre identité. Techniquement, ce partage repose sur un rôle PostgreSQL de groupe (`sillon_tutoriel_lecteurs`) créé par ce `postinst`, dont `creer_utilisateur()` (`schema.sql`, `sillon-server` $\geq 0.1.30$) rend automatiquement membre tout nouveau compte.

`sudo apt-get purge sillon-tutoriel` supprime intégralement le compte `demo`, sa base, le rôle de partage et le PDF publié ; une réinstallation repart proprement à zéro.

- `outils/peupler_demo.py` (script autonome, hors paquet) : alternative plus légère pour un jeu de données fictif (`communes_exemple`), sans tutoriel PDF ni partage automatique avec les autres comptes :

```
python3 outils/peupler_demo.py --url https://<adresse-du-serveur> --admin-
password '<mot de passe administrateur>' --demo-password '<mot de passe
choisi>'
```

Le mot de passe du compte `demo@sillon.local` créé par `sillon-tutoriel` est **généré aléatoirement**, comme celui du compte administrateur (§5) : écrit dans `/root/sillon-demo-initial.txt` (lecture root uniquement, jamais affiché en clair dans les journaux d’installation), et à conserver uniquement pour un accès manuel exceptionnel (dépannage, réimport) — les agents n’ont normalement jamais besoin de s’authentifier avec ce compte, le jeu de données leur étant partagé directement. `outils/peupler_demo.py` reste un script volontaire (jamais installé automatiquement) dont le mot de passe par défaut (`--demo-password, demo`) doit être changé explicitement avant tout usage sur une VM exposée.

Jeu de données massif (sillon-demo-sirene)

Sixième paquet `.deb`, optionnel, complémentaire de `sillon-tutoriel` (mêmes règles : jamais en production, dérogation assumée). Il télécharge et importe le fichier Sirene « StockEtablissement » complet (INSEE, ~43,9 millions de lignes, Licence Ouverte 2.0), puis dépose un second jeu de scripts Python/R démontrant les possibilités de l’environnement d’exécution à cette échelle (cahier des charges §12.8).

C’est le seul paquet du projet qui a besoin d’un accès Internet réel sur la machine cible au moment de son installation — tous les autres composants sont vendorisés (§4). Deux façons de procéder :

- **Automatique** (cas courant, VM disposant d’une sortie Internet) :

```
sudo dpkg -i sillon-demo-sirene_0.1.0_all.deb
```

Le `postinst` télécharge lui-même l’archive (environ 2,9 Go), l’extraît et la réduit aux colonnes utiles (environ 3,8 Go de CSV, quelques heures selon la machine — la sortie du `postinst` progresse par paliers de 5 millions de lignes), puis l’importe via l’API d’import normale.
- **Manuelle** (machine cible sans sortie Internet directe, ou pour ne pas dépendre de la durée du téléchargement pendant l’installation) : télécharger au préalable, depuis une machine qui a accès à Internet, le fichier « Sirene : Fichier StockEtablissement » (pas la variante *Historique*, pas le format *parquet*) depuis data.gouv.fr

(jeu de données « Base Sirene des entreprises et de leurs établissements »), puis :

```
sudo mkdir -p /var/lib/sillon-demo-sirene
sudo cp stock-stocketablissement-csv.zip
/var/lib/sillon-demo-sirene/stock_etablissement.zip
sudo dpkg -i sillon-demo-sirene_0.1.0_all.deb
```

Le postinst détecte l'archive déjà présente et saute le téléchargement, pour passer directement à l'extraction puis à l'import. Le même mécanisme de reprise s'applique si une précédente tentative a déjà produit le fichier réduit (`sirene_reduit.csv` dans ce même répertoire) : `dpkg -i` peut être relancé sans redemander le fichier ni recommencer l'extraction en cas d'échec à une étape ultérieure (constaté en pratique lors des tests de ce paquet).

Prévoir au moins **12 Go d'espace disque libre** sur la partition qui héberge `/var/lib` le temps de l'installation (archive + fichier réduit + tampon d'import), même si l'espace définitivement occupé par la table importée est moindre.

`sudo apt-get purge sillon-demo-sirene` ne supprime que son répertoire de travail temporaire : la donnée elle-même (une table de plus dans la base du compte demo) disparaît avec `sillon-tutoriel`, quel que soit l'ordre de purge entre les deux paquets.

7.3 Quotas et paramètres applicatifs

Les quotas (taille maximale d'import CSV, durée maximale d'un job, CPU/RAM/PID par conteneur, quota disque par base, etc.) sont stockés dans la table `parametres` du catalogue applicatif, modifiable sans redémarrage de service ni reconstruction de paquet — soit depuis le panneau d'administration de l'application, soit directement en SQL :

```
UPDATE public.parametres SET valeur = '4096' WHERE cle = 'taille_max_csv_mo';
```

Valeurs par défaut à l'installation :

Paramètre	Valeur par défaut
<code>taille_max_csv_mo</code>	2048
<code>seuil_import_synchrone_mo</code>	10
<code>duree_max_job_minutes</code>	30
<code>cpu_max_conteneur_vcpu</code>	2
<code>ram_max_conteneur_mo</code>	4096
<code>jobs_simultanes_par_utilisateur</code>	4
<code>quota_disque_base_mo</code>	20480
<code>work_mem_default_mo</code>	64
<code>maintenance_work_mem_default_mo</code>	256
<code>connexions_max_par_utilisateur</code>	5

Import volumineux lent à indexer : chaque import CSV crée automatiquement un index par colonne (§7.4 du cahier des charges — GIN trigram pour le texte, B-tree pour les dates), coûteux en mémoire de travail pour un fichier de plusieurs millions de lignes. `maintenance_work_mem_default_mo` (appliqué par utilisateur, comme `work_mem_default_mo`) contrôle cette mémoire — la relever nettement (par exemple 1024 à 2048 selon la RAM disponible sur le serveur) accélère sensiblement la construction des index pour un import de cette taille :

```
UPDATE public.parametres SET valeur = '2048' WHERE cle =
'maintenance_work_mem_default_mo';
ALTER ROLE "<role_pg_de_l_utilisateur>" SET maintenance_work_mem = '2097152'; --
```

en kilo-octets, $2048 * 1024$

Cette seconde commande ne s'applique qu'aux comptes déjà créés (`role_pg` visible dans `public.utilisateurs`) : modifier le paramètre seul suffit pour tout compte créé après coup. `sillon-demo-sirene` applique déjà cette optimisation automatiquement à son installation pour le compte de démonstration (§7.2).

Identifier les requêtes les plus coûteuses : `sillon-server` active `pg_stat_statements` à l'installation (§7.1) — consultable directement en SQL, par exemple pour les dix requêtes cumulant le plus de temps d'exécution :

```
SELECT query, calls, round(total_exec_time) AS temps_total_ms,
round(mean_exec_time) AS temps_moyen_ms
FROM public.pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10;
```

7.4 Purge automatique des fichiers et données accumulés (`sillon-purge`)

Sans intervention, trois choses grossissent indéfiniment sur le serveur au fil de l'usage : les résultats de jobs (exports SQL, archives de scripts, §5.3/§5.4 du cahier des charges), les dépôts temporaires d'import CSV jamais validés — orphelins laissés par un import interrompu en cours de route — et le journal d'audit (§8.12). Le paquet optionnel `sillon-purge` couvre les trois, installé après `sillon-server` ($\geq 0.1.32$) :

```
sudo dpkg -i sillon-purge_0.1.0_all.deb
```

Il livre un seul point d'entrée, `/etc/cron.d/sillon_purge`, exécuté chaque jour à 3 h du matin en tant que `root` (nécessaire pour atteindre les fichiers de résultats appartenant à l'UID remappé du conteneur d'exécution, §7.7 du cahier des charges — les opérations sur le catalogue applicatif, elles, sont déléguées à `sudo -u postgres psql`, comme le reste des scripts de maintenance de SILLON). Aucun service à surveiller : c'est une tâche cron, pas un démon.

Durées de rétention, modifiables à chaud comme les autres paramètres applicatifs (§7.3 ci-dessus — panneau d'administration ou SQL direct), sans redémarrage de service ni reconstruction de paquet :

Paramètre	Valeur par défaut	Porte sur
<code>retention_resultats_jours</code>	90	<code>/var/lib/sillon/resultats</code>
<code>retention_staging_heures</code>	48	<code>/var/lib/sillon/staging</code> (dépôts orphelins uniquement — un import mené à son terme normal est déjà nettoyé par le code applicatif lui-même, indépendamment de cette purge)
<code>retention_audit_mois</code>	12	<code>public.audit_logs</code>

```
UPDATE public.parametres SET valeur = '180' WHERE cle =
'retention_resultats_jours';
```

Suivi : la sortie de chaque exécution est envoyée au journal système plutôt qu'à un fichier dédié sous `/var/log` (qui redeviendrait lui-même un fichier à purger) :

```
journalctl -t sillon-purge -n 50
```

Avant `sillon-purge` (`sillon-server` $< 0.1.32$) : ces mêmes durées étaient réparties dans deux `/etc/cron.d` livrés directement par `sillon-server` (résultats de jobs et journal d'audit uniquement — pas de purge des dépôts orphelins), avec des valeurs codées en dur plutôt que modifiables via `parametres`. Une mise à niveau de `sillon-server` vers 0.1.32 ou ultérieur retire proprement ces deux anciens fichiers ; installer ensuite `sillon-purge` pour retrouver une purge active.

8. Désinstallation

Deux niveaux, sémantique standard `dpkg` :

- `remove` : arrête les services et retire les fichiers du paquet, mais conserve `/etc/sillon/secrets.env`, `/etc/sillon-api.conf` et la base `sillon_catalog` — une réinstallation ultérieure reprend alors à l'identique (mêmes secrets, mêmes comptes), sans passer par la branche « premier déploiement ».

```
sudo apt-get remove sillon-purge sillon-image-execution sillon-worker  
sillon-orchestrateur sillon-server
```

- `purge` : en plus de ce qui précède, supprime réellement le catalogue applicatif (base `sillon_catalog`, rôles PostgreSQL personnels et de service qu'elle porte) ainsi que `/etc/sillon`, `/etc/sillon-api.conf`, `/etc/ssl/sillon` et `/var/lib/sillon` — une réinstallation ultérieure repart alors intégralement à zéro (nouveaux secrets, nouveau compte administrateur).

```
sudo apt-get purge sillon-purge sillon-image-execution sillon-worker sillon-  
orchestrateur sillon-server
```

`sillon-purge` dépend de `sillon-server` : retirer `sillon-server` sans le lister également fait échouer `apt-get remove/purge` (ou entraîne sa suppression automatique avec avertissement, selon le gestionnaire de paquets utilisé) — l'inclure explicitement dans la commande évite toute ambiguïté. Il ne porte aucune donnée propre (ni base, ni fichier hors son unique `/etc/cron.d/sillon_purge`) : `remove` et `purge` ont pour lui un effet strictement identique, retrait pur et simple du cron de purge.

Dans les deux cas, **les bases de travail créées par les agents** (§4.4 du cahier des charges — une base physique par agent, distincte du catalogue `sillon_catalog`) **ne sont jamais supprimées automatiquement**, ni par `remove` ni par `purge` : ce sont des données applicatives, pas des artefacts d'installation. Pour les retirer explicitement si c'est réellement souhaité :

```
sudo -u postgres psql -c "\l" | grep sillon_  
sudo -u postgres dropdb <nom_de_la_base>
```

Cas particulier de `sillon-tutoriel` : contrairement aux quatre autres paquets, son `purge` supprime intégralement le compte `demo@sillon.local`, sa base, le rôle de partage (`sillon_tutoriel_lecteurs`, retiré de tous les comptes qui en étaient membres) et le PDF publié — c'est un compte de démonstration jetable, pas une donnée d'agent réel, la même prudence ne s'applique pas.

```
sudo apt-get purge sillon-tutoriel
```

Rappel : aucun mécanisme de sauvegarde n'est porté par l'application — la protection contre la perte de données relève de l'infrastructure d'hébergement de la direction.

9. Dépannage courant

Symptôme	Cause probable	Vérification
Import CSV volumineux rejeté (413)	Anciennement un défaut Nginx (<code>client_max_body_size</code>) — corrigé, la limite Nginx est illimitée et seule la limite applicative (<code>taille_max_csv_mo</code>) s'applique	<code>grep client_max_body_size /etc/nginx/sites-available/sillon</code>
Script utilisateur ne démarre jamais	Session D-Bus de <code>www-data</code> non encore prête après un redémarrage	<code>systemctl status user@33.service</code> , puis <code>systemctl restart sillon-worker</code>

Symptôme	Cause probable	Vérification
	du serveur	
sillon-orchestrateur refuse de s'installer	sillon-server non installé/configuré au préalable (/etc/sillon/secrets.env absent)	Installer/vérifier sillon-server d'abord
Requêtes SQL normales, mais tous les scripts échouent (statut « erreur ») après un changement d'adresse IP ou un redémarrage du serveur	Adresse hôte détectée par le postinst de sillon-worker obsolète (§9.1)	journalctl -u sillon-worker -n 50, rechercher une erreur de connexion PostgreSQL
Bouton « Journal » (onglet Suivi) affiche systématiquement « journal vide pour l'instant », même pour un job en cours	Corrigé (python3 -u / stdbuf, worker.py) : la sortie standard d'un interprète non attaché à un terminal était mise en tampon par blocs, jamais vidée avant la fin d'un script court — le journal réellement produit n'apparaissait qu'une fois le job déjà terminé (et donc déjà uniquement dans l'archive téléchargeable, plus dans la fenêtre « en cours »)	sudo dpkg -i sillon-worker_0.1.0_all.deb puis systemctl restart sillon-worker si une version antérieure au correctif est installée
Import « depuis data.gouv.fr » échoue avec « URL de ressource non autorisée »	Depuis la version 0.1.6 de sillon-orchestrateur, seules les URL http(s):// dont l'adresse résolue est publique (ni privée/RFC 1918, ni loopback, ni lien-local) sont acceptées — protection contre le détournement de cet import vers un fichier local ou un service interne (SSRF, cf. rapport d'audit de sécurité). Pas de restriction par nom de domaine : les ressources data.gouv.fr sont hébergées sur des domaines tiers très divers (portails opendata régionaux, ArcGIS...), une restriction à data.gouv.fr casserait la quasi-totalité des imports réels — testé et corrigé en ce sens (§9, non-régression VM de test). Ce message ne devrait donc apparaître que pour une ressource pointant réellement vers une adresse privée/interne	journalctl -u sillon-orchestrateur -n 50

9.1 Changement d'adresse IP du serveur (ou redémarrage avec IP dynamique)

Symptôme observé en pratique : après un changement d'adresse IP (DHCP, réaffectation de VM, changement d'interface réseau) suivi ou non d'un redémarrage, les requêtes SQL de l'onglet Travaux continuent de fonctionner normalement, mais **tous** les scripts Python/R déposés échouent immédiatement (statut « erreur », sans même produire de journal d'exécution).

Cause : les scripts s'exécutent dans un conteneur Podman rootless, qui n'a pas d'accès direct à localhost de l'hôte. Le réseau rootless par défaut (pasta) route l'alias host.containers.internal vers l'hôte réel, mais en réemettant la connexion avec l'adresse IP LAN de l'hôte comme adresse source (jamais 127.0.0.1, constaté en pratique dans les journaux PostgreSQL) — c'est pourquoi le postinst de sillon-worker autorise explicitement **cette seule adresse**, détectée une

fois à l'installation, à la fois dans `pg_hba.conf` (qui peut l'accepter) :

```
host      all             all             <adresse détectée à l'installation>/32
scram-sha-256
```

et dans `listen_addresses` de `postgresql.conf` (sur quelle(s) interface(s) PostgreSQL écoute réellement — *depuis la version 0.1.2 de `sillon-worker`, restreint à `localhost` + cette seule adresse, plutôt qu'à * (toutes les interfaces) auparavant, pour ne pas exposer PostgreSQL au-delà du strict nécessaire — voir rapport d'audit de sécurité) :

```
listen_addresses = 'localhost,<adresse détectée à l'installation>'
```

Cette détection n'est **jamais refaite automatiquement** par la suite. Si l'adresse IP du serveur change après coup, ces deux réglages deviennent obsolètes : PostgreSQL n'écoute même plus sur la nouvelle adresse (donc `pg_hba.conf` n'entre jamais en jeu), donc tout script (qui doit s'y connecter via `SILLON_DSN`) échoue dès sa tentative de connexion — alors que les requêtes SQL de l'onglet Travaux, elles, transitent par l'orchestrateur en `host=localhost` et ne sont jamais concernées.

Remédiation — deux options équivalentes :

- **Réinstaller** `sillon-worker` (le plus simple, redétecte l'adresse courante, ajoute la règle `pg_hba.conf` manquante et met à jour `listen_addresses` en conséquence, avec redémarrage de PostgreSQL si nécessaire — un simple reload ne suffit pas pour `listen_addresses`, cf. §7.1) :

```
sudo dpkg -i sillon-worker_0.1.2_all.deb
```

L'ancienne règle `pg_hba.conf` (adresse obsolète) n'est pas supprimée — elle ne gêne pas, `pg_hba.conf` accepte plusieurs règles `host` sans conflit — mais peut être retirée manuellement par propreté si souhaité.

`listen_addresses`, lui, est remplacé (pas cumulé) par la nouvelle adresse.

- **Corriger à la main**, sans reconstruire/redéployer de paquet — **deux réglages à modifier**, pas seulement `pg_hba.conf` :

```
ADRESSE_HOTE=$(ip -4 route get 1.1.1.1 | grep -oP 'src \K\S+' | head -1)
echo "host      all             all             ${ADRESSE_HOTE}/32
scram-sha-256" \
| sudo tee -a /etc/postgresql/17/main/pg_hba.conf
sudo sed -i "s/^#\?listen_addresses.*/listen_addresses = 'localhost,$
{ADRESSE_HOTE}'/" \
/etc/postgresql/17/main/postgresql.conf
sudo systemctl restart postgresql
```

Le `restart` (pas un simple `reload`) est nécessaire ici : `listen_addresses` est à contexte « `postmaster` », seul un redémarrage rouvre les sockets d'écoute (§7.1).

Point de vigilance pour un serveur à adresse IP dynamique (DHCP sans bail réservé) : ce mécanisme n'est pas auto-réparant — chaque changement d'adresse casse à nouveau les scripts jusqu'à la prochaine réinstallation ou correction manuelle. Pour un déploiement durable, préférer une **adresse IP statique ou un bail DHCP réservé** pour le serveur SILLON, ce qui élimine le problème à la source plutôt que de le corriger à chaque occurrence.

Autres points de friction possibles liés à un changement d'adresse IP (moins critiques, sans impact constaté à ce jour) :

- Le certificat TLS auto-signé est généré avec `CN=localhost` (aucune adresse IP ni nom d'hôte spécifique) : un changement d'IP ne l'invalidé pas, mais l'avertissement de certificat non reconnu par le navigateur reste présent quelle que soit l'adresse utilisée pour se connecter — sans lien avec ce changement.
- Si `SILLON_URL` a été configuré manuellement (`systemctl edit sillon-orchestrateur` **et** `systemctl edit sillon-worker`,

§7.1 — les deux services le lisent chacun de leur côté) avec une adresse IP littérale plutôt qu'un nom de domaine, les liens inclus dans les notifications par mail pointeront vers l'ancienne adresse après un changement — à corriger manuellement dans ce cas précis, sur les deux services.

9.2 Tous les scripts échouent après un redémarrage du serveur, sans changement d'adresse IP

Distinct du §9.1 : ici l'adresse IP du serveur n'a pas changé, mais un **redémarrage complet de la machine** (pas un simple `systemctl restart`) suffit à provoquer le même symptôme (scripts en erreur, requêtes SQL normales) — de façon intermittente, pas à chaque redémarrage.

Cause : sur une interface réseau configurée en DHCP via `allow-hotplug` (cas le plus courant sur une VM, `/etc/network/interfaces`), l'attribution de l'adresse IP se fait de façon asynchrone par rapport au reste de la séquence de démarrage — `network-online.target` peut être atteint **avant** la fin du bail DHCP (constaté en pratique : jusqu'à une dizaine de secondes d'écart). Si PostgreSQL démarre dans cette fenêtre, il ne parvient à se lier qu'à `localhost` (l'adresse LAN de `listen_addresses`, §7.7, n'existe pas encore sur l'interface à cet instant) — sans erreur bloquante, juste un avertissement dans son propre journal (n'a pas pu lier IPv4 à l'adresse ... : Ne peut attribuer l'adresse demandée). Les scripts, qui doivent joindre PostgreSQL depuis leur conteneur via cette adresse LAN, échouent alors jusqu'au prochain redémarrage manuel de PostgreSQL — l'application web, elle, continue de fonctionner normalement (elle passe par `localhost`).

Corrigé depuis `sillon-worker 0.1.3` : le paquet installe un [`drop-in systemd`](#) sur le gabarit `postgresql@.service` (celui réellement instancié en `postgresql@<version>-main.service` — `postgresql.service` lui-même n'est qu'une unité factice sur Debian, `ExecStart=/bin/true`, à ne pas confondre) qui fait patienter le démarrage de PostgreSQL jusqu'à ce qu'une route de sortie IPv4 soit utilisable (jusqu'à 30 secondes, puis démarrage normal même si le délai est dépassé — un problème réseau ne doit jamais empêcher PostgreSQL de démarrer). Validé par un redémarrage complet réel de la VM de test : sans le correctif, PostgreSQL ne se liait qu'à `localhost` ; avec, il patiente ~20 secondes le temps du bail DHCP puis se lie correctement aux deux adresses.

Sur une installation déjà à jour, rien à faire : ce correctif s'applique automatiquement à l'installation/mise à jour de `sillon-worker`. Pour une installation antérieure à la version 0.1.3, sans reconstruire de paquet, poser le *drop-in* à la main :

```
sudo mkdir -p /etc/systemd/system/postgresql@.service.d
sudo tee /etc/systemd/system/postgresql@.service.d/sillon-worker-attendre-
reseau.conf <<'EOF'
[Service]
ExecStartPre=/opt/sillon-worker/attendre_reseau.sh
EOF
sudo systemctl daemon-reload
```

(nécessite que `/opt/sillon-worker/attendre_reseau.sh` existe déjà — livré par le paquet `sillon-worker` depuis la même version ; sur une version antérieure, mettre à jour le paquet plutôt que de recréer ce script à la main).

Comme pour le §9.1 : une adresse IP statique ou un bail DHCP réservé réduit encore la fenêtre de risque (la négociation DHCP, la partie la plus lente de la séquence, disparaît), mais le correctif ci-dessus reste la protection de fond — il fonctionne quel que soit le temps que prend la configuration réseau à ce démarrage précis.